SafeCharge-RL

Reinforcement learning for EV charging when drivers' departure times are wrong or misreported

Code snippets with explanations: the RL core and everything set up around it. Code is copied verbatim from the repo, with real line numbers.

Team 32	Adithyan R (AM.SC.U4AIE23065), Adithya Rajesh (AM.SC.U4AIE23007), Balu M Krishna (AM.SC.U4AIE23026)
Guide	Dr. Geetha M, Dept. of CSE (AI), Amrita Vishwa Vidyapeetham
Prepared	9 Oct 2026, for the mid-review (9-10 Oct). Code: github.com/adithyx-0/safecharge-rl (private repo)

How to read this. Each block shows real code, then a table that explains it line by line (the numbers on the left are real line numbers in the file, so you can open the file and follow). Long lines are wrapped for print. Read in order: the setup (data, config) first, then the environment, the safety layer, the network, the algorithm, and last the evaluation code.

Contents

- A. Setup: data cleaning, price curve, configs, report model
- B. The environment (the MDP): reset, view, observation, step
- C. The safety projection
- D. The baselines, the offline optimum and MPC
- E. The neural network (Deep Sets actor and critic)
- F. PPO-Lagrangian: advantages, dual step, rollout, update, warm start
- G. Training loop and evaluation code (train.py, metrics, paired replay)

A. Setup

A1. Price curve (src/safecharge/data.py, lines 25-31)

```
25  def price_curve(date, tariff="real"):
26      """Price per step ($/kWh) for a weekday. tariff: 'real' (summer Jun-Sep, else winter) | 'summer'
            (always summer)."""
27      summer = tariff == "summer" or (tariff == "real" and 6 <= date.month <= 9)
28      vals = _SUMMER if summer else _WINTER
29      hour = np.arange(T) * DELTA
30      idx = np.searchsorted(_HOURS, hour, side="right") - 1
31      return np.asarray(vals)[idx]
```

Lines	What it does
27	summer is True if tariff is 'summer' (used everywhere in our experiments) or a real June-September date
28	pick the 5 summer or winter prices
29	hour of each of the 96 steps: 0, 0.25, 0.5 ...
30	searchsorted maps each hour to its price period (periods start at hours 0, 8, 12, 18, 23)
31	returns a 96-long array of $/kWh

Remember: Step 50 is hour 12.5, which falls in the 12-18 period, so its price is $0.26668 (the peak).

A2. Cleaning sessions into days (the core of build_days) (src/safecharge/data.py, lines 97-131)

```
 97      d = df[(df["kwh"] > 0)]
 98      stats["zero_energy"] = len(df) - len(d)
 99      ok = d["disc"] > d["conn"]
100      stats["invalid_range"] = int((~ok).sum())
101      d = d[ok].copy()
102      d["date"] = d["conn"].dt.normalize()
103      cross = d["disc"].dt.normalize() > d["date"]
104      stats["crosses_midnight"] = int(cross.sum())
105      if midnight == "drop":
106          d = d[~cross]
107      d["a"] = np.ceil((d["conn"] - d["date"]) / pd.Timedelta(minutes=15)).astype(int)
108      dep = np.floor((d["disc"] - d["date"]) / pd.Timedelta(minutes=15)).astype(int)
109      d["b"] = np.minimum(dep, T)  # 'clip' mode: departure clipped to day end
110      keep = d["b"] > d["a"]
111      stats["empty_window"] = int((~keep).sum())
112      d = d[keep]
113      if weekdays_only:
114          d = d[d["date"].dt.dayofweek < 5]
115      cap = P_PORT * (d["b"] - d["a"]) * DELTA
116      stats["energy_clipped"] = int((d["kwh"] > cap + 1e-9).sum())
117      stats["kwh_recorded"] = float(d["kwh"].sum())
118      d["rep_step"] = (d["req"] - d["date"]) / pd.Timedelta(minutes=15)
119      d["e"] = np.minimum(d["kwh"], cap)
120      stats["kwh_after_clip"] = float(d["e"].sum())
121      days = []
122      for date, g in d.groupby("date"):
123          tz = g["tz"].iloc[0]
124          if is_dst_day(date, tz):
125              stats["dst_day"] += 1
126              continue
127          g = g.sort_values(["a", "b"])
128          days.append(Day(date.date(), g["a"].to_numpy(), g["b"].to_numpy(), g["e"].to_numpy(),
129                          price_curve(date, tariff), g["sessionID"].tolist(), g["kwh"].to_numpy(),
130                          g["rep_step"].to_numpy(dtype=float)))
131      return days, stats
```

Lines	What it does
97-101	keep sessions with energy above 0 and disconnect after connect
102-106	day = local date of arrival; a session whose disconnect date is later crosses midnight, and with midnight='drop' it is removed (6.3% Caltech)
107	arrival step = ceil(minutes since midnight / 15): rounded UP
108-109	departure step = floor(...): rounded DOWN, capped at 96
110-112	drop sessions whose window became empty
113-114	weekdays only
115	cap = most energy a port can deliver in the stay = 6.656 kW x steps x 0.25 h
119	e = min(recorded kWh, cap): the energy the car needs

Lines	What it does
121-130	group by date, skip clock-change days, sort by arrival, and build one Day object per date (arrival, departure, energy, price, ids, recorded kWh, recorded driver report)

Remember: Rounding arrival up and departure down means the simulated stay is never longer than the real one. Every exclusion is counted in the stats dict so the share affected can be reported.

A3. Splits and cache (src/safecharge/config.py, lines 16-32)

```
16  def get_days(cfg, split, site=None, raw_dir="data/raw", cache_dir="data/processed"):
17      """Days of `split` ('train'|'val'|'test'|'post') for `site` (default cfg['site']), already filtered
            per cfg."""
18      site = site or cfg["site"]
19      key = f"{site}_{cfg['midnight']}_{cfg['tariff']}_{'-'.join(cfg['exclude_stations'].get(site, []))}.pkl"
20      path = os.path.join(cache_dir, key)
21      if os.path.exists(path):
22          with open(path, "rb") as f:
23              days = pickle.load(f)
24      else:
25          df = load_sessions(os.path.join(raw_dir, f"{site}_sessions.json"))
26          days, _ = build_days(df, midnight=cfg["midnight"], tariff=cfg["tariff"],
27                               exclude_stations=cfg["exclude_stations"].get(site, []))
28          os.makedirs(cache_dir, exist_ok=True)
29          with open(path, "wb") as f:
30              pickle.dump(days, f)
31      a, b = (pd.Timestamp(x).date() for x in cfg["splits"][split])
32      return [d for d in days if a <= d.date <= b]
```

Lines	What it does
19-20	cache file name depends on site, midnight rule, tariff and excluded stations
21-23	if cached, load the saved list of days
24-30	else read the raw JSON, build the days, save them
31-32	return the days whose date lies inside the split's date range (train, val, test or post)

Remember: Splits are chronological. Data come from data/raw (not in GitHub) and the cache goes to data/processed.

A4. Settings: base.yaml, derived yaml and e1.yaml

```
3  site: caltech
4  exclude_stations:            # Caltech station 2-39-81-4550 is a ~46 kW fast charger, not a 6.656 kW port
       (54 modelled ports)
5    caltech: ["2-39-81-4550"]
6    jpl: []
7  midnight: drop               # sessions crossing local midnight are dropped (6.3% Caltech, 1.45% JPL)
8  tariff: summer               # SCE TOU-EV-4 summer weekday curve applied to ALL weekdays (a controlled
       assumption, not the real Jun-Sep-only tariff)
9  splits:                      # inclusive local dates; chronological, one pricing regime, pre-COVID
10   train: ["2019-01-01", "2019-08-31"]
11   val:   ["2019-09-01", "2019-10-31"]
12   test:  ["2019-11-01", "2020-02-29"]
13   post:  ["2021-01-01", "2021-09-30"]
```

```
1  # generated by scripts/calibrate.py from TRAIN days only; do not edit by hand
2  P_grid: 34.61
3  P_peak: 173.06
4  d1: 0.7133
5  d2: 2.923
6  e_ref: 8.359
7  kappa: 0.2
8  n_train_days: 174
9  offline_penalty_M: 5.0
10 reward_scale: 4.78
11 site: caltech
```

Setting	Meaning
exclude_stations / midnight / tariff	The data choices (fast station out, midnight crossers dropped, summer tariff on all weekdays)
splits	Chronological train, validation, test, and a post-COVID set
P_grid 34.61, P_peak 173.06	Grid cap = 0.20 x typical full-rate peak, computed from TRAIN days only
e_ref 8.359	Mean energy per car; unmet energy is divided by it (cost 1)

Setting	Meaning
d1 0.7133, d2 2.923	Budgets. d1 = 2% of requested energy per day in cost units; d2 = mean wear proxy of the FCFS/EDF/LLF baselines
reward_scale 4.78	Divides the dollar cost so a typical day's return is about -10
offline_penalty_M 5.0	Penalty per kWh unmet in the LP (about 19x the peak price)

```
 8  agent:
 9    hid: 64
10    use_count: true          # occupied-port count appended to the pooled context (set false for the literal
          design)
11    pool: mean               # design: masked mean-pool. ablations: meanmax, attn
12    lr: 0.0003
13    lr_final: 0.1            # linear decay to 10% of lr
14    n_envs: 48               # episodes (days) per update
15    epochs: 10
16    minibatch: 256
17    clip: 0.2
18    gamma: 0.999
19    gae_lambda: 0.97
20    ent_coef: 0.001
21    target_kl: 0.02
22    dual_lr: 0.05            # on clipped relative violation J/d - 1
23    dual_clip: [-1, 1]
24    lam_max: 10.0
25    j_ema: 0.7
26    critic_warmup: 30
27    bc: {expert: LLF, epochs: 200}
28    scheme: uniform          # projection scheme (E5: priority)
29    train_reports:
30      - {mode: M0}
31  train:
32    n_updates: 1500
33    eval_every: 20
```

Setting	Meaning
hid 64, use_count true, pool mean	Network width; add the occupied-port count to the pooled context; masked mean-pool (the design)
lr 3e-4, lr_final 0.1	Learning rate decays linearly to 10%
n_envs 48, epochs 10, minibatch 256, clip 0.2	48 days per update, 10 passes, PPO clip range
gamma 0.999, gae_lambda 0.97	Discount (design says 0.99; changed and disclosed) and GAE smoothing
ent_coef 0.001, target_kl 0.02	Small exploration bonus; stop an update early if the policy moves too far
dual_lr 0.05, dual_clip, lam_max 10, j_ema 0.7	Multiplier step size, clip of the relative violation, cap on lambda, smoothing of the cost estimate
critic_warmup 30, bc {LLF, 200 epochs}	Train only the critic for 30 updates; behaviour-cloning warm start from LLF (not in the design)
scheme uniform	Projection scheme (E5 would use priority)
train_reports [M0]	Train with truthful reports (E4 configs use mixtures)
n_updates 1500, eval_every 20	Training length; validation check every 20 updates

A5. Report model (src/safecharge/report_model.py, lines 12-15)

```
12  def base_draws(seed, day_index, n):
13      rng = np.random.default_rng([seed, day_index])
14      return dict(eps=rng.uniform(-1, 1, n), late=rng.uniform(0, 1, n), sel=rng.uniform(0, 1, n),
            imp=rng.uniform(0, 1, n))
15
```

Lines	What it does
13	a random generator seeded by (seed, day index): the same for every method and every mode
14	four arrays of uniform numbers: eps (error), late, sel (who is selected), imp (imputation)

Remember: These fixed draws are why every method sees identical reports, and why larger w or rho reuse the same numbers.

A6. Making the reported departure (src/safecharge/report_model.py, lines 31-49)

```
31      n = len(day.arr)
32      dep = day.dep.astype(float)
33      d = base_draws(seed, day_index, n)
34      if mode == "M0":
35          rep = dep.copy()
36      elif mode == "M2":
37          rep = dep + d["eps"] * w / DELTA
38      elif mode == "M3":
39          rep = dep + np.where(d["sel"] < q, d["late"] * w / DELTA, 0.0)
40      elif mode == "M4":
41          rep = dep - np.where(d["sel"] < rho, s / DELTA, 0.0)
42      elif mode == "M1r":
43          imputed = dep + pool[np.minimum((d["imp"] * len(pool)).astype(int), len(pool) - 1)]
44          rep = np.where(np.isnan(day.rep_rec), imputed, day.rep_rec)
45      else:
46          raise ValueError(f"unknown mode {mode!r}")
47      if focal is not None:
48          rep[focal] = dep[focal] - focal_s / DELTA
49      return np.maximum(rep, day.arr.astype(float))
```

Lines	What it does
32	true departure as float steps
34-35	M0: report = true departure
36-37	M2: add eps x w/DELTA (w in hours, DELTA = 0.25 h per step)
38-39	M3: for a share q of cars (sel below q) add a late amount up to w hours
40-41	M4: for a share rho of cars subtract s hours (cheating early)
42-44	M1r: recorded driver report where it exists, else true departure plus an error from the real error pool
47-48	paired replay hook: force one focal car to report dep minus focal_s
49	never earlier than arrival

Remember: The vehicle always leaves at its true time; only the reported time changes.

B. The environment (the MDP)

B1. reset (src/safecharge/env.py, lines 27-42)

```
27      def reset(self, seed=None, options=None):
28          super().reset(seed=seed)
29          self.day, self.rep = options["day"], np.asarray(options["rep"], dtype=float)
30          n = len(self.day.arr)
31          self.t = 0
32          self.vid = -np.ones(self.N, int)                # vehicle index per port, -1 = empty
33          self.r = np.zeros(self.N)
34          self.E = np.zeros(n)                            # delivered energy per vehicle
35          self.m = np.zeros(n)                            # unmet energy at true departure
36          self.dropped = np.zeros(n, bool)                # no free port on arrival
37          self.next_arr = 0
38          self.last_power = 0.0
39          self.power_log = np.zeros(T)
40          self.port_power_log = []                        # (t, vehicle, kW) rows with p>0, for metrics
41          self._plug_in()
42          return self._obs(), {}
```

Lines	What it does
29	the day (true times) and the reported departures come in through options
32	vid: which vehicle is on each port (-1 = empty)
33	r: remaining energy per port
34-36	E: delivered energy per vehicle; m: unmet energy per vehicle; dropped: no free port on arrival
37-40	bookkeeping: next arrival, last total power, power log, port power log
41-42	plug in cars arriving at step 0 and return the first observation

B2. view: the only information a policy may use (src/safecharge/env.py, lines 58-69)

```
58      def view(self):
59          """What a policy / rule baseline may use: reported (never true) departure, per port."""
60          occ = self.vid >= 0
61          v = np.where(occ, self.vid, 0)
62          tau = np.where(occ, np.maximum(0.0, self.rep[v] - self.t) * DELTA, 0.0)
63          r = np.where(occ, self.r, 0.0)
64          lax = np.where(occ, tau - r / self.P_port, 0.0)
65          h = occ * np.minimum(self.P_port, r / DELTA)
66          if self.curtail:
67              h = h * (self.t < np.ceil(np.where(occ, self.rep[v], 0.0)))
68          arr = np.where(occ, self.day.arr[v], 0)
69          return dict(occ=occ, r=r, tau=tau, laxity=lax, h=h, arr=arr, rep=np.where(occ, self.rep[v], 0.0))
```

Lines	What it does
60-61	occupied ports; vehicle index (0 for empty so indexing works)
62	tau = time left until the REPORTED departure, floored at 0, in hours
63-64	remaining energy r and laxity = tau - r / P_port (spare time at full power)
65	h = max useful power: min(port max, energy that would just finish the car in one step)
66-67	curtail mode (E4): no power once the reported departure has passed
69	returns only reported-time quantities; the true departure is never here

Remember: A test checks that changing only the true departure does not change what the policy sees.

B3. Observation (src/safecharge/env.py, lines 71-77)

```
71      def _obs(self):
72          w = self.view()
73          ports = np.stack([w["occ"].astype(float), w["r"] / OBS_R, w["tau"] / OBS_TAU, w["laxity"] /
                OBS_TAU], 1)
74          price = self.day.price[min(self.t, T - 1)]
75          ang = 2 * np.pi * self.t / T
76          g = [price / OBS_PRICE, np.sin(ang), np.cos(ang), self.last_power / self.P_grid]
77          return np.concatenate([ports.ravel(), g])
```

Lines	What it does
73	per port: [occupied, r/30, tau/24, laxity/24]: four features, scaled to about unit size
74-76	globals: price/0.27, sin and cos of time of day, last total power / grid cap

Lines	What it does
77	flattened: N x 4 port numbers + 4 globals = 4N + 4 (220 numbers for 54 ports)

B4. step: one 15-minute decision (src/safecharge/env.py, lines 80-116)

```
 80      def step(self, u):
 81          w = self.view()
 82          h = w["h"]
 83          p_tilde = np.clip(np.asarray(u, dtype=float), 0.0, 1.0) * h
 84          weights = 1.0 + np.maximum(w["laxity"], 0.0)  # used only by scheme='priority' (PROVISIONAL weighting)
 85          p = project(p_tilde, h, self.P_grid, self.scheme, weights)
 86
 87          occ = self.vid >= 0
 88          dE = p * DELTA
 89          self.r = np.maximum(self.r - dE, 0.0)
 90          self.r[self.r < 1e-9] = 0.0
 91          np.add.at(self.E, self.vid[occ], dE[occ])
 92          total = float(p.sum())
 93          self.power_log[self.t] = total
 94          for k in np.flatnonzero(p > 0):
 95              self.port_power_log.append((self.t, self.vid[k], p[k]))
 96          price = self.day.price[self.t]
 97          dollars = price * total * DELTA
 98
 99          # departures at the end of step t
100          c1 = 0.0
101          for k in np.flatnonzero(occ):
102              i = self.vid[k]
103              if self.day.dep[i] == self.t + 1:
104                  self.m[i] = self.r[k]
105                  c1 += self.m[i]
106                  self.vid[k], self.r[k] = -1, 0.0
107          c1 /= self.e_ref
108          c2 = float(np.sum(p > 0.8 * self.P_port)) / self.N
109
110          self.last_power = total
111          self.t += 1
112          done = self.t >= T
113          if not done:
114              self._plug_in()
115          info = dict(cost1=c1, cost2=c2, dollars=dollars, power=total, p=p, p_tilde=p_tilde, price=price)
116          return self._obs(), -dollars / self.reward_scale, done, False, info
```

Lines	What it does
83	p_tilde = u x h: the requested power per port
85	project(): cut the requests so the total fits the grid cap
88-91	remaining energy r falls by p x 0.25 h; delivered energy E rises
92-95	log total power and per-port power (used by metrics)
96-97	dollars = price x total power x 0.25 h
101-106	cars whose TRUE departure is this step leave: unmet energy m = remaining r; cost 1 adds it; the port is freed
107	cost 1 divided by e_ref
108	cost 2 = share of ports above 80% of max power
110-114	advance time; plug in new arrivals; done after step 96
116	returns observation, reward = -dollars / reward_scale, done flag, info with costs

Remember: Everything the design defines is here: state, action, reward, two costs, episode. The projection sits inside step(), so the grid cap cannot be broken.

C. The safety projection

C1. project (src/safecharge/projection.py, lines 10-30)

```
10  def project(p_tilde, h, P_grid, scheme="uniform", weights=None, iters=60):
11      p_tilde = np.asarray(p_tilde, dtype=float)
12      h = np.asarray(h, dtype=float)
13      p0 = np.clip(p_tilde, 0.0, h)
14      if p0.sum() <= P_grid:
15          return p0
16      if scheme == "uniform":
17          w = np.ones_like(p0)
18      elif scheme == "priority":
19          w = np.asarray(weights, dtype=float)
20      else:
21          raise ValueError(f"unknown projection scheme {scheme!r}")
22      # sum(clip(p~ - mu*w, 0, h)) is continuous and non-increasing in mu; it is 0 once mu*w >= p~ for all w>0
23      lo, hi = 0.0, float(np.max(p0 / np.maximum(w, 1e-12)))
24      for _ in range(iters):
25          mid = 0.5 * (lo + hi)
26          if np.clip(p_tilde - mid * w, 0.0, h).sum() > P_grid:
27              lo = mid
28          else:
29              hi = mid
30      return np.clip(p_tilde - hi * w, 0.0, h)  # hi side is always feasible
```

Lines	What it does
13	clip requests to [0, h]
14-15	if the total is already within the cap, return unchanged (identity)
16-21	uniform: every port is cut by the same mu (w = 1). priority: cut mu x weight, so urgent cars are cut less
23	mu is searched between 0 and the largest request, so the total at the top end is 0
24-29	bisection, 60 halvings: if the total at mid still exceeds the cap, raise the lower end; else lower the upper end
30	return the result at the upper end, which is always feasible

Remember: The total is continuous and decreasing in mu, so bisection always finds the smallest mu that fits. Example: cap 20, requests 12/10/8 (total 30) become 8.67/6.67/4.67 with mu = 3.33.

D. Baselines, offline optimum and MPC

D1. Rule baselines (src/safecharge/baselines.py, lines 8-27)

```
 8  def rule_action(env, kind):
 9      w = env.view()
10      h, occ = w["h"], w["occ"]
11      if kind == "FCFS":
12          key = w["arr"].astype(float)
13      elif kind == "EDF":
14          key = w["rep"]
15      elif kind == "LLF":
16          key = w["laxity"]
17      else:
18          raise ValueError(f"unknown rule {kind!r}")
19      order = [k for k in np.argsort(key, kind="stable") if occ[k]]
20      p = np.zeros(env.N)
21      left = env.P_grid
22      for k in order:
23          p[k] = min(h[k], left)
24          left -= p[k]
25          if left <= 0:
26              break
27      return np.divide(p, h, out=np.zeros(env.N), where=h > 0)
```

Lines	What it does
9-10	the same view the policy gets: reported times only
11-17	sort key: arrival (FCFS), reported departure (EDF) or laxity (LLF)
19	order occupied ports by the key (smallest first)
20-26	greedy: give each port its maximum power in that order until the grid cap is used up
27	return u = p / h so the environment treats it like any action

Remember: Rules see exactly what the learned policy sees, so the comparison is fair.

D2. Offline optimum (hindsight LP) (src/safecharge/offline_lp.py, lines 15-41)

```
15  def solve_offline(day, P_grid, P_port, M, vehicles=None):
16      idx = np.arange(len(day.arr)) if vehicles is None else np.asarray(vehicles)
17      n = len(idx)
18      cols, veh_of, t_of = 0, [], []
19      for j, i in enumerate(idx):
20          for t in range(day.arr[i], day.dep[i]):
21              veh_of.append(j)
22              t_of.append(t)
23      nx = len(veh_of)
24      veh_of, t_of = np.array(veh_of, int), np.array(t_of, int)
25      c = np.concatenate([day.price[t_of] * DELTA, np.full(n, M)])
26      # grid rows (inequality)
27      A_ub = coo_matrix((np.ones(nx), (t_of, np.arange(nx))), shape=(T, nx + n)).tocsr()
28      b_ub = np.full(T, P_grid)
29      # energy rows (equality): sum_t x*DELTA + m_j = e_j
30      rows = np.concatenate([veh_of, np.arange(n)])
31      colsx = np.concatenate([np.arange(nx), nx + np.arange(n)])
32      vals = np.concatenate([np.full(nx, DELTA), np.ones(n)])
33      A_eq = coo_matrix((vals, (rows, colsx)), shape=(n, nx + n)).tocsr()
34      b_eq = day.energy[idx]
35      bounds = [(0, P_port)] * nx + [(0, None)] * n
36      res = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method="highs")
37      if not res.success:
38          raise RuntimeError(f"offline LP failed: {res.message}")
39      x, m = res.x[:nx], res.x[nx:]
40      cost = float(np.sum(day.price[t_of] * x * DELTA))
41      return dict(cost=cost, unmet=float(m.sum()), objective=float(res.fun), m=m, x=x, veh=veh_of, t=t_of,
            idx=idx)
```

Lines	What it does
16-24	one variable x[i,t] for each car and each step between its arrival and TRUE departure
25	cost vector: price x 0.25 for charging, plus M for each kWh unmet
26-28	grid constraint per step: sum of x at step t at most P_grid
29-34	energy balance per car: sum of x x 0.25 + unmet = needed energy
35	bounds: 0 to port max for x, unmet at least 0
36-38	solve with HiGHS (about 3 ms per day)

Lines	What it does
39-41	return cost, unmet energy, the objective and the schedule

Remember: It knows the true departures and the whole day, so it is a lower bound. The experiment script asserts that no method beats it on any day.

D3. MPC (re-plan every step) (src/safecharge/mpc.py, lines 16-41)

```
16  def mpc_action(env, M=5.0):
17      w = env.view()
18      occ = np.flatnonzero(w["occ"])
19      if len(occ) == 0:
20          return np.zeros(env.N)
21      t0 = env.t
22      price = env.day.price
23      rep_end = np.clip(np.ceil(w["rep"][occ]).astype(int), t0 + 1, T)   # last allowed step is rep_end - 1;
            at least the current step
24      cols = [(j, t) for j, k in enumerate(occ) for t in range(t0, rep_end[j])]
25      j_of, t_of = np.array([c[0] for c in cols]), np.array([c[1] for c in cols])
26      nx, n = len(cols), len(occ)
27      c = np.concatenate([price[t_of] * DELTA, np.full(n, M)])
28      A_ub = coo_matrix((np.ones(nx), (t_of - t0, np.arange(nx))), shape=(T - t0, nx + n)).tocsr()
29      A_eq = coo_matrix((np.concatenate([np.full(nx, DELTA), np.ones(n)]),
30                         (np.concatenate([j_of, np.arange(n)]), np.concatenate([np.arange(nx), nx +
                               np.arange(n)]))), shape=(n, nx + n)).tocsr()
31      res = linprog(c, A_ub=A_ub, b_ub=np.full(T - t0, env.P_grid), A_eq=A_eq, b_eq=w["r"][occ],
32                    bounds=[(0, env.P_port)] * nx + [(0, None)] * n, method="highs")
33      u = np.zeros(env.N)
34      if not res.success:
35          return u
36      now = t_of == t0
37      p = np.zeros(n)
38      p[j_of[now]] = res.x[:nx][now]
39      h = w["h"][occ]
40      u[occ] = np.divide(p, h, out=np.zeros(n), where=h > 0)
41      return np.clip(u, 0, 1)
```

Lines	What it does
18-20	only the cars plugged in now
23	last allowed step for each car = its REPORTED departure (at least the current step)
24-27	variables and costs as in the offline LP, but only from now on
28-32	grid constraint per remaining step, energy balance with the car's remaining energy, solve
33-41	take only the first step of the plan and convert it to u = p / h

Remember: MPC trusts the declared deadlines. With wrong reports it postpones charging and runs out of time (about 30x more unmet energy at w = 2 h).

E. The neural network (Deep Sets)

E1. Masked pooling (src/safecharge/agents/deepsets.py, lines 30-35)

```
30  def pool(z, occ, n_ports, use_count, kind="mean"):
31      cnt = occ.sum(-1, keepdim=True)
32      ctx = (z * occ.unsqueeze(-1)).sum(-2) / cnt.clamp(min=1.0)
33      if kind == "meanmax":     # ablation: also the per-feature max over occupied ports (sees the most
            urgent vehicle)
34          ctx = torch.cat([ctx, torch.where(occ.unsqueeze(-1) > 0, z, torch.full_like(z,
                -1e4)).max(-2).values.clamp(min=-1.0)], -1)
35      return torch.cat([ctx, cnt / n_ports], -1) if use_count else ctx
```

Lines	What it does
31	cnt: number of occupied ports
32	context = average of the port embeddings over occupied ports only (empty ports are masked out)
33-34	optional ablation: also take the max over ports
35	append the occupied count / N so the network knows how many cars compete

Remember: Averaging makes the network independent of port order and of how many cars there are.

E2. Actor (the policy) (src/safecharge/agents/deepsets.py, lines 52-86)

```
52  class Actor(nn.Module):
53      def __init__(self, n_ports, hid=64, use_count=True, init_log_std=-0.5, pool_kind="mean"):
54          super().__init__()
55          self.n, self.use_count, self.pool_kind = n_ports, use_count, pool_kind
56          self.enc = mlp([4, hid, hid])
57          self.attn = SelfAttn(hid) if pool_kind == "attn" else None
58          ctx_dim = hid * (2 if pool_kind == "meanmax" else 1) + int(use_count)
59          self.head = mlp([hid + ctx_dim + 4, hid, 2], last_gain=0.01)
60          self.init_log_std = init_log_std
61
62      def forward(self, obs):
63          ports, g, occ = split_obs(obs, self.n)
64          z = torch.tanh(self.enc(ports)) * occ.unsqueeze(-1)
65          if self.attn is not None:
66              z = self.attn(z, occ)
67          ctx = pool(z, occ, self.n, self.use_count, self.pool_kind)
68          h = self.head(torch.cat([z, ctx.unsqueeze(-2).expand(*z.shape[:-1], ctx.shape[-1]),
69                                   g.unsqueeze(-2).expand(*z.shape[:-1], g.shape[-1])], -1))
70          mean = h[..., 0]
71          log_std = (self.init_log_std + h[..., 1]).clamp(-4.0, 1.0)
72          return mean, log_std, occ
73
74      def act(self, obs, deterministic=False):
75          mean, log_std, occ = self(obs)
76          a = mean if deterministic else mean + torch.randn_like(mean) * log_std.exp()
77          return a, self.log_prob(a, mean, log_std, occ)
78
79      @staticmethod
80      def log_prob(a, mean, log_std, occ):
81          lp = -0.5 * ((a - mean) / log_std.exp()) ** 2 - log_std - 0.9189385332046727
82          return (lp * occ).sum(-1)
83
84      @staticmethod
85      def entropy(log_std, occ):
86          return ((log_std + 1.4189385332046727) * occ).sum(-1)
```

Lines	What it does
56	shared encoder: 4 features to 64 to 64, applied to every port
59	shared head: input = port embedding + context + 4 globals; output = 2 numbers per port; tiny last layer so the policy starts near 'half power'
63-67	split the observation, encode each port, zero out empty ports, pool into a context
68-69	concatenate (port embedding, context, globals) for every port and run the head
70-71	mean of the Gaussian and a log standard deviation (start -0.5, clamped to [-4, 1])
74-77	act: sample a = mean + noise x std (or the mean when testing); the applied action is sigmoid(a) in [0,1]
80-82	log-probability summed over occupied ports only
85-86	entropy over occupied ports

Remember: Parameters are shared across ports, so the policy works for 5 or 45 cars. A test checks it is permutation-equivariant and ignores empty ports.

E3. Critic (the value function) (src/safecharge/agents/deepsets.py, lines 89-105)

```
 89  class Critic(nn.Module):
 90      """Outputs 3 values in normalised units; the algorithm multiplies by its running per-head scale."""
 91
 92      def __init__(self, n_ports, hid=64, use_count=True, n_heads=3, pool_kind="mean"):
 93          super().__init__()
 94          self.n, self.use_count, self.pool_kind = n_ports, use_count, pool_kind
 95          self.enc = mlp([4, hid, hid])
 96          self.attn = SelfAttn(hid) if pool_kind == "attn" else None
 97          ctx_dim = hid * (2 if pool_kind == "meanmax" else 1) + int(use_count)
 98          self.head = mlp([ctx_dim + 4, 2 * hid, 2 * hid, n_heads], last_gain=1.0)
 99
100      def forward(self, obs):
101          ports, g, occ = split_obs(obs, self.n)
102          z = torch.tanh(self.enc(ports)) * occ.unsqueeze(-1)
103          if self.attn is not None:
104              z = self.attn(z, occ)
105          return self.head(torch.cat([pool(z, occ, self.n, self.use_count, self.pool_kind), g], -1))
```

Lines	What it does
95-98	same encoder; head takes the pooled context + globals and outputs 3 values
100-105	the 3 values = expected discounted reward, expected unmet-energy cost, expected wear cost

F. PPO-Lagrangian

F1. GAE and the dual step (src/safecharge/agents/ppo_lag.py, lines 21-36)

```
21  def gae(rew, val, gamma, lam):
22      """rew, val: (T, K); val has T+1 rows (bootstrap 0 at episode end). Returns advantages (T, K) and
            returns."""
23      adv = np.zeros_like(rew)
24      last = 0.0
25      for t in reversed(range(rew.shape[0])):
26          delta = rew[t] + gamma * val[t + 1] - val[t]
27          last = delta + gamma * lam * last
28          adv[t] = last
29      return adv, adv + val[:-1]
30
31
32  def dual_step(lam, J, d, alpha, clip=(-1.0, 1.0), lam_max=np.inf):
33      """lam_k <- clip_to[0, lam_max](lam_k + alpha * clip(J_k / d_k - 1)). The relative violation is
            clipped so that a huge early
34      violation does not wind lam up (it took about 1800 updates to unwind lam = 89 in the first run);
            lam_max caps the weight on the costs."""
35      v = np.clip(np.asarray(J) / np.asarray(d) - 1.0, clip[0], clip[1])
36      return np.clip(lam + alpha * v, 0.0, lam_max)
```

Lines	What it does
22-29	advantage = weighted sum of future TD errors, computed backwards through the 96 steps; returns = advantage + value
32-36	dual step: relative violation J/d - 1, clipped to [-1, 1]; lambda moves by alpha x that, never below 0 and never above lam_max

Remember: Lambda goes up when a cost is above its budget and down when it is below.

F2. Rollout (collect) (src/safecharge/agents/ppo_lag.py, lines 65-84)

```
65      def collect(self, train_days):
66          K, N = self.cfg["n_envs"], self.N
67          idx = self.rng.choice(len(train_days), K, replace=len(train_days) < K)
68          obs = np.stack([env.reset(options=dict(day=train_days[i], rep=self._episode_reports(train_days[i],
                int(i))))[0]
69                          for env, i in zip(self.envs, idx)])
70          O, A, LP = np.zeros((T, K, obs.shape[1]), np.float32), np.zeros((T, K, N), np.float32),
                np.zeros((T, K), np.float32)
71          R, C1, C2 = (np.zeros((T, K)) for _ in range(3))
72          for t in range(T):
73              o = torch.as_tensor(obs, dtype=torch.float32)
74              with torch.no_grad():
75                  a, lp = self.actor.act(o)
76              u = torch.sigmoid(a).numpy().astype(np.float64)
77              O[t], A[t], LP[t] = o.numpy(), a.numpy(), lp.numpy()
78              nxt = []
79              for k, env in enumerate(self.envs):
80                  ob, r, done, _, info = env.step(u[k])
81                  R[t, k], C1[t, k], C2[t, k] = r, info["cost1"], info["cost2"]
82                  nxt.append(ob)
83              obs = np.stack(nxt)
84          return dict(O=O, A=A, LP=LP, R=R, C=np.stack([C1, C2]))
```

Lines	What it does
66-67	K = 48 episodes in lockstep
67	choose 48 random training days (with replacement only if there are fewer than 48)
68-69	reset each env with its day and a report draw from the training mixture
70-71	buffers for observations, actions, log-probs, rewards and the 2 costs
73-77	for each of 96 steps: the actor samples a and its log-prob, u = sigmoid(a)
78-83	step every env, store reward and both costs
84	return the batch

F3. Update, part 1: advantages for three signals (src/safecharge/agents/ppo_lag.py, lines 94-115)

```
 94      def update(self, b):
 95          cfg, g, lm = self.cfg, self.cfg["gamma"], self.cfg["gae_lambda"]
 96          T_, K = b["R"].shape
 97          O = torch.as_tensor(b["O"].reshape(T_ * K, -1))
 98          with torch.no_grad():
 99              v = (self.critic(O).numpy() * self.scale).reshape(T_, K, 3)
100          sig = [b["R"], b["C"][0], b["C"][1]]
101          advs, rets = [], []
102          for h in range(3):
103              val = np.concatenate([v[:, :, h], np.zeros((1, K))], 0)
104              a_, r_ = gae(sig[h], val, g, lm)
105              advs.append(a_), rets.append(r_)
106          # critic scales: EMA of the batch return std (floor), then targets in normalised units
107          for h in range(3):
108              self.scale[h] = 0.9 * self.scale[h] + 0.1 * max(float(rets[h].std()), 1e-2) if self.n_updates
                     else max(float(rets[h].std()), 1e-2)
109          nadv = [(a_ - a_.mean()) / (a_.std() + 1e-8) for a_ in advs]
110          lam = self.lam
111          comb = (nadv[0] - lam[0] * nadv[1] - lam[1] * nadv[2]) / (1.0 + lam.sum())
112          tens = lambda x: torch.as_tensor(np.asarray(x).reshape(T_ * K, *np.asarray(x).shape[2:]),
                 dtype=torch.float32)
113          A_, LP_, ADV = tens(b["A"]), tens(b["LP"]), tens(comb)
114          RET = torch.as_tensor(np.stack([r_.reshape(-1) for r_ in rets], 1) / self.scale, dtype=torch.float32)
115          n, mb = T_ * K, cfg["minibatch"]
```

Lines	What it does
98-99	critic values for every state, multiplied by running scales
100-105	GAE separately for reward, cost 1 and cost 2
107-108	running scale of each signal (moving average of the return std)
109	normalise each advantage to mean 0, std 1
110-111	combined advantage = (A_reward - lam1 x A_cost1 - lam2 x A_cost2) / (1 + lam1 + lam2)
114	critic targets in normalised units

Remember: A large lambda makes the policy treat that cost like a big penalty.

F4. Update, part 2: PPO step and dual ascent (src/safecharge/agents/ppo_lag.py, lines 117-156)

```
117         for _ in range(cfg["epochs"]):
118             perm = torch.randperm(n)
119             stop = False
120             for s in range(0, n, mb):
121                 i = perm[s:s + mb]
122                 mean, log_std, occ = self.actor(O[i])
123                 lp = Actor.log_prob(A_[i], mean, log_std, occ)
124                 ratio = (lp - LP_[i]).exp()
125                 pl = -torch.min(ratio * ADV[i], ratio.clamp(1 - cfg["clip"], 1 + cfg["clip"]) * ADV[i]).mean()
126                 ent = Actor.entropy(log_std, occ).mean()
127                 if self.n_updates >= cfg.get("critic_warmup", 0):
128                     self.opt_a.zero_grad()
129                     (pl - cfg["ent_coef"] * ent).backward()
130                     torch.nn.utils.clip_grad_norm_(self.actor.parameters(), 0.5)
131                     self.opt_a.step()
132                 vl = ((self.critic(O[i]) - RET[i]) ** 2).mean()
133                 self.opt_c.zero_grad()
134                 vl.backward()
135                 torch.nn.utils.clip_grad_norm_(self.critic.parameters(), 0.5)
136                 self.opt_c.step()
137                 kl = float((LP_[i] - lp.detach()).mean())
138                 for key, val in (("pl", float(pl)), ("vl", float(vl)), ("kl", kl), ("ent", float(ent))):
139                     stats[key] += val
140                 stats["cnt"] += 1
141                 if kl > 1.5 * cfg["target_kl"]:
142                     stop = True
143                     break
144             if stop:
145                 break
146         # dual ascent on smoothed undiscounted episode costs
147         J = b["C"].sum(1).mean(-1)                          # (2,) undiscounted episode cost (sum over
                time), mean over the K episodes
148         self.J_ema = J if self.J_ema is None else cfg["j_ema"] * self.J_ema + (1 - cfg["j_ema"]) * J
149         if self.n_updates >= cfg.get("critic_warmup", 0):
150             self.lam = dual_step(self.lam, self.J_ema, self.d, cfg["dual_lr"], tuple(cfg.get("dual_clip",
                    (-1.0, 1.0))), cfg.get("lam_max", np.inf))
151         self.n_updates += 1
152         c = max(stats.pop("cnt"), 1)
153         out = {k: v / c for k, v in stats.items()}
154         out.update(J1=float(J[0]), J2=float(J[1]), lam1=float(self.lam[0]), lam2=float(self.lam[1]),
155                    ep_return=float(b["R"].sum(0).mean()))
156         return out
```

Lines	What it does
117-121	10 epochs of shuffled minibatches of 256 samples
122-125	new log-prob, probability ratio, PPO clipped objective (clip 0.2)
126	entropy bonus
127-131	actor step with gradient clipping 0.5, only after the critic warm-up
132-136	critic regression to the targets
137-145	if the policy moved too far (KL), stop early
147-150	average undiscounted cost per day, smoothed, then dual_step updates the two lambdas
151-156	increment counter; return statistics for the log (costs, lambdas, return)

Remember: This is the whole algorithm: collect days, compute three advantages, take a PPO step on the lambda-weighted advantage, then update lambda from the measured costs.

F5. Behaviour-cloning warm start (not in the submitted design) (src/safecharge/agents/ppo_lag.py, lines 159-181)

```
159      def bc_pretrain(self, train_days, expert="LLF", epochs=30, lr=1e-3, batch=512):
160          """Regress the actor mean onto logit(u_expert) on the expert's own trajectories (train days,
                 configured report mixture)."""
161          env, O, U = ChargingEnv(**self.env_kw), [], []
162          for i, d in enumerate(train_days):
163              obs, _ = env.reset(options=dict(day=d, rep=self._episode_reports(d, i)))
164              for _ in range(T):
165                  u = mpc_action(env) if expert == "MPC" else rule_action(env, expert)
166                  O.append(obs), U.append(u)
167                  obs, _, done, _, _ = env.step(u)
168          O, U = torch.as_tensor(np.array(O), dtype=torch.float32), torch.as_tensor(np.array(U),
                 dtype=torch.float32)
169          tgt = torch.logit(U.clamp(0.05, 0.95))
170          opt = torch.optim.Adam(self.actor.parameters(), lr=lr)
171          loss = None
172          for _ in range(epochs):
173              perm = torch.randperm(len(O))
174              for s_ in range(0, len(O), batch):
175                  i = perm[s_:s_ + batch]
176                  mean, _, occ = self.actor(O[i])
177                  loss = (((mean - tgt[i]) ** 2) * occ).sum() / occ.sum().clamp(min=1)
178                  opt.zero_grad()
179                  loss.backward()
180                  opt.step()
181          return float(loss)
```

Lines	What it does
161-167	play the training days with LLF (or MPC) and record (observation, action) pairs
168-169	convert expert actions to logits (clamped to 0.05-0.95)
170-181	regress the actor's mean onto those logits, ignoring empty ports

Remember: Without this the policy stayed worse than every rule after 2,000 updates. Say it is an addition when asked.

G. Training loop and evaluation code

G1. train.py: setup and loop (scripts/train.py, lines 38-50)

```
38  agent = PPOLag(cfg["agent"], N_PORTS[site], der["P_grid"], der["e_ref"], der["reward_scale"], (der["d1"],
        der["d2"]), seed=args.seed)
39  agent.pool = empirical_errors(train)
40  last, best = os.path.join(run, "last.pt"), os.path.join(run, "best.pt")
41  bc_cfg = cfg["agent"].get("bc")
42  log_path = os.path.join(run, "log.csv")
43  rows, best_key = [], (9, 1e9)
44  if os.path.exists(last):
45      agent.load(torch.load(last, weights_only=False))
46      rows = pd.read_csv(log_path).to_dict("records") if os.path.exists(log_path) else []
47      best_key = tuple(torch.load(best, weights_only=False)["key"]) if os.path.exists(best) else best_key
48      print(f"resumed at update {agent.n_updates}")
49  elif bc_cfg:
50      print(f"behaviour-cloning warm start from {bc_cfg['expert']}: final loss {agent.bc_pretrain(train,
            **bc_cfg):.4f}")
```

Lines	What it does
38	create the agent with P_grid, e_ref, reward_scale and the two budgets from the derived config
39	pool of real report errors (for M1r)
44-48	resume from last.pt if it exists
49-50	otherwise run the behaviour-cloning warm start

G2. train.py: main loop (scripts/train.py, lines 61-78)

```
61  t0 = time.time()
62  while agent.n_updates < n_updates:
63      agent.set_lr(agent.n_updates / n_updates)
64      batch = agent.collect(train)
65      row = dict(update=agent.n_updates + 1, **agent.update(batch))
66      if agent.n_updates % cfg["train"]["eval_every"] == 0 or agent.n_updates == n_updates:
67          vs, key = val_summary()
68          row.update(vs)
69          if key < best_key:
70              best_key = key
71              torch.save(dict(**agent.state(), key=key), best)
72          torch.save(agent.state(), last)
73          print(f"[{agent.n_updates:4d}] ret={row['ep_return']:7.2f} J1={row['J1']:.3f}/{der['d1']:.2f}
                J2={row['J2']:.2f}/{der['d2']:.2f} "
74                f"lam=({row['lam1']:.2f},{row['lam2']:.2f}) | val U={vs['val_U']:.4f}
                      $/kWh={vs['val_cost_per_kwh']:.4f} "
75                f"J1={vs['val_J1']:.3f} J2={vs['val_J2']:.2f} gridviol={vs['val_grid_viol']} |
                      {time.time()-t0:.0f}s", flush=True)
76      rows.append(row)
77      pd.DataFrame(rows).to_csv(log_path, index=False)
78  torch.save(agent.state(), last)
```

Lines	What it does
63	decay the learning rate
64-65	collect a batch and update
66-72	every 20 updates: evaluate on validation days, save best.pt if feasible and cheapest, save last.pt
73-77	print progress and write the CSV log

G3. Metrics for one day (src/safecharge/metrics.py, lines 34-59)

```
34  def day_metrics(env, cost1_sum=np.nan, cost2_sum=np.nan, ms_per_step=np.nan):
35      d = env.day
36      e = d.energy
37      n = len(e)
38      E = env.E
39      kwh = float(E.sum())
40      dollars = float(np.sum(d.price * env.power_log * DELTA))
41      s = E / e
42      log = np.array(env.port_power_log).reshape(-1, 3)
43      active = len(log)
44      high = int(np.sum(log[:, 2] > 0.8 * env.P_port)) if active else 0
45      strat = _strat(d, E)
46      out = dict(
47          date=str(d.date), n_veh=n, dropped=int(env.dropped.sum()),
48          cost=dollars, kwh=kwh, cost_per_kwh=dollars / kwh if kwh > 0 else np.nan,
49          unmet_share=float(np.maximum(e - E, 0).sum() / e.sum()),
50          miss_rate=float(np.mean(E < 0.95 * e)),
51          grid_viol=int(np.sum(env.power_log > env.P_grid + 1e-9)),
52          grid_max_excess=float(max(0.0, np.max(env.power_log - env.P_grid))),
53          high_power_ratio=high / active if active else np.nan,
54          jain=float(s.sum() ** 2 / (n * np.sum(s ** 2))) if np.sum(s ** 2) > 0 else np.nan,
55          cost1_sum=cost1_sum, cost2_sum=cost2_sum, ms_per_step=ms_per_step,
56          peak_power=float(env.power_log.max()),
57      )
58      out.update(strat)
59      return out
```

Lines	What it does
40	dollars = sum price x power x 0.25 h
41	s = delivered / requested per car
49	unmet share = total unmet / total requested
50	miss rate = share of cars with under 95% of their request
51-52	grid violations: count of steps above the cap, and the largest excess
53	high-power ratio: share of charging port-steps above 80% of max power (wear proxy)
54	Jain fairness (sum s)^2 / (n x sum s^2): 1 means everyone is served equally

G4. Paired replay: the cheating measure (src/safecharge/metrics.py, lines 70-81)

```
70  def paired_gain(env, day, base_rep, act_fn, focal, s):
71      """Misreporting gain for vehicle `focal`: same day, same policy, same other reports; honest vs
            reporting dep - s hours.
72      Returns (gain_kwh, gain_pct_of_e). act_fn must be deterministic (or reseeded identically) for a clean
            pairing."""
73      rep_honest = base_rep.copy()
74      rep_honest[focal] = max(float(day.dep[focal]), float(day.arr[focal]))
75      rep_lie = base_rep.copy()
76      rep_lie[focal] = max(float(day.dep[focal]) - s / DELTA, float(day.arr[focal]))
77      _, e1 = rollout(env, day, rep_honest, act_fn)
78      E_honest = e1.E[focal]
79      _, e2 = rollout(env, day, rep_lie, act_fn)
80      g = e2.E[focal] - E_honest
81      return float(g), float(100 * g / day.energy[focal])
```

Lines	What it does
73-74	run 1: the focal car reports its true departure
75-76	run 2: it reports s hours earlier
77-80	same day, same policy, same other reports; gain = energy in run 2 minus run 1
81	returned in kWh and as % of the car's request

Remember: With s = 0 the gain is exactly 0 (a test). Everything else is held fixed, so any difference is caused by the report alone.

G5. Running a day with any policy (src/safecharge/metrics.py, lines 11-22)

```
11  def rollout(env, day, rep, act_fn):
12      """Run one day. act_fn(env, obs) -> u. Returns (metrics dict, env)."""
13      obs, _ = env.reset(options=dict(day=day, rep=rep))
14      done, c1, c2, steps, t0 = False, 0.0, 0.0, 0, time.perf_counter()
15      while not done:
16          u = act_fn(env, obs)
17          obs, _, done, _, info = env.step(u)
18          c1 += info["cost1"]
19          c2 += info["cost2"]
20          steps += 1
21      ms = (time.perf_counter() - t0) / steps * 1000
22      return day_metrics(env, c1, c2, ms), env
```

Lines	What it does
13	reset with the day and its reports
15-20	loop: act, step, add the two costs
21-22	time per step and the day's metrics

Remember: Rules, MPC and the learned policy all go through this one function.